

# Lab 8: RISC Programs with Input/Output

Name: **Loc D. T. Nguyen**

Student ID: **24125093**

April 12, 2026

## 1 Exercise 3: String to Integer Conversion (atoi)

### 1.1 Overview

This exercise implements an `atoi` function to convert an ASCII string stored in data memory into a signed integer. The program is capable of handling both positive and negative values by checking for a leading hyphen ('-') and processes numerical digits until a space or newline character is encountered.

### 1.2 Algorithm and Implementation

The string-to-integer conversion relies on extracting the integer value of each character by subtracting 48 (the ASCII value of '0'). The running total is multiplied by 10 for each new digit parsed. Since the basic Logisim MIPS processor lacks a native multiplication instruction, multiplication by 10 is handled via logical left shifts and addition:

$$x \times 10 = (x \ll 3) + (x \ll 1)$$

Listing 1: Snippet: Multiply by 10 and Accumulate

```
# $2 contains current total, $9 contains new extracted digit
addiu $9, $9, -48    # Convert ASCII to integer (c - '0')
sll $10, $2, 3       # $10 = result * 8
sll $11, $2, 1       # $11 = result * 2
add $2, $10, $11     # result = result * 10
add $2, $2, $9       # Add current digit to total
```

### 1.3 Simulation Observations

- The test program successfully utilizes the `readString` subroutine to capture user input from the keyboard mapping (0x80000004), echoing it back to the console mapping (0x80000008).
- The generated integer in register `$2` is correctly subtracted from the secret integer (-357) loaded from data memory to trigger the appropriate `bltz` or `beq` branching logic.

## 2 Exercise 4: Hardware Control via MMIO (1ed)

### 2.1 Overview

This program demonstrates write-access to hardware peripherals mapped to system memory. A specific LED (from 0 to 9) on the Logisim circuit board is toggled based on an integer input collected from the TTY console.

### 2.2 Memory-Mapped I/O Concept

The LEDs are connected to the Southbridge and mapped to the special I/O address 0x8000000C. To toggle a specific LED without overwriting the state of the others, the program must load the current state word, apply a bitwise XOR mask, and store the word back to the MMIO address. The hardware state updates instantly upon the `sw` execution.

Listing 2: Snippet: LED Toggling Logic via XOR

```
led:
    lui $8, 0x8000
    ori $8, $8, 0x000C # Base LED MMIO Address: 0x8000000C
    lw $9, 0($8)       # Read current state of all LEDs
    li $10, 1
    sllv $10, $10, $4   # Shift a '1' by n positions (dynamic shift)
    xor $9, $9, $10     # Toggle the specific bit n
    sw $9, 0($8)       # Write the updated state back to LEDs
    jr $31
```

## 3 Exercise 5: Caesar Cipher Data Encryption

### 3.1 Overview

The final exercise synthesizes string parsing and mathematical operations to encrypt a user-provided string using a Caesar Cipher. The shift factor  $n$  and the plaintext string are captured via keyboard polling, utilizing `readWithoutEcho` to prevent displaying the plaintext secret on the screen.

### 3.2 Encryption Logic

The algorithm shifts each valid lowercase alphabetical character ('a' to 'z') by  $n$  positions. The code performs boundary checks to ensure it wraps around the alphabet correctly. Mathematically, for a lowercase character  $c$ , the shift operation enforces:

$$c_{new} \in [97, 122]$$

In assembly, instead of modulo arithmetic, this is achieved using conditional branches and repeated addition or subtraction if the shifted ASCII value falls out of the bounds.

Listing 3: Snippet: Cipher Wrap-Around Logic

```
caesar_check_over:
    li $t0, 122          # ASCII 'z'
    ble $t1, $t0, caesar_check_under
    addiu $t1, $t1, -26   # Wrap backwards into the alphabet
    j caesar_check_over  # Re-evaluate
```

### 3.3 Simulation Observations

- Reading the shift value  $n$  required combining `readString` and `atoi`, storing the final result in a preserved register (\$12) before executing the string capture.
- The `readWithoutEcho` loop successfully terminates on a newline (0x0A) and writes a null terminator to memory, which is critical for the subsequent `printString` call to output the ciphertext cleanly.

## 4 Conclusion

In this lab, advanced concepts of the MIPS RISC instruction set were explored, focusing heavily on subroutines and Memory-Mapped I/O (MMIO). Implementing the `atoi` function demonstrated how to manipulate ASCII representations into logical mathematical bases using purely arithmetic instructions. Exercise 4 illustrated direct hardware manipulation by reading, masking, and writing to the LED register at 0x8000000C. Finally, the Caesar cipher exercise required managing multiple pointers and subroutine returns (`jal` and `jr $31`) within a single executable, proving the processor's capability to handle complex branching, hardware polling, and memory overwrites simultaneously.